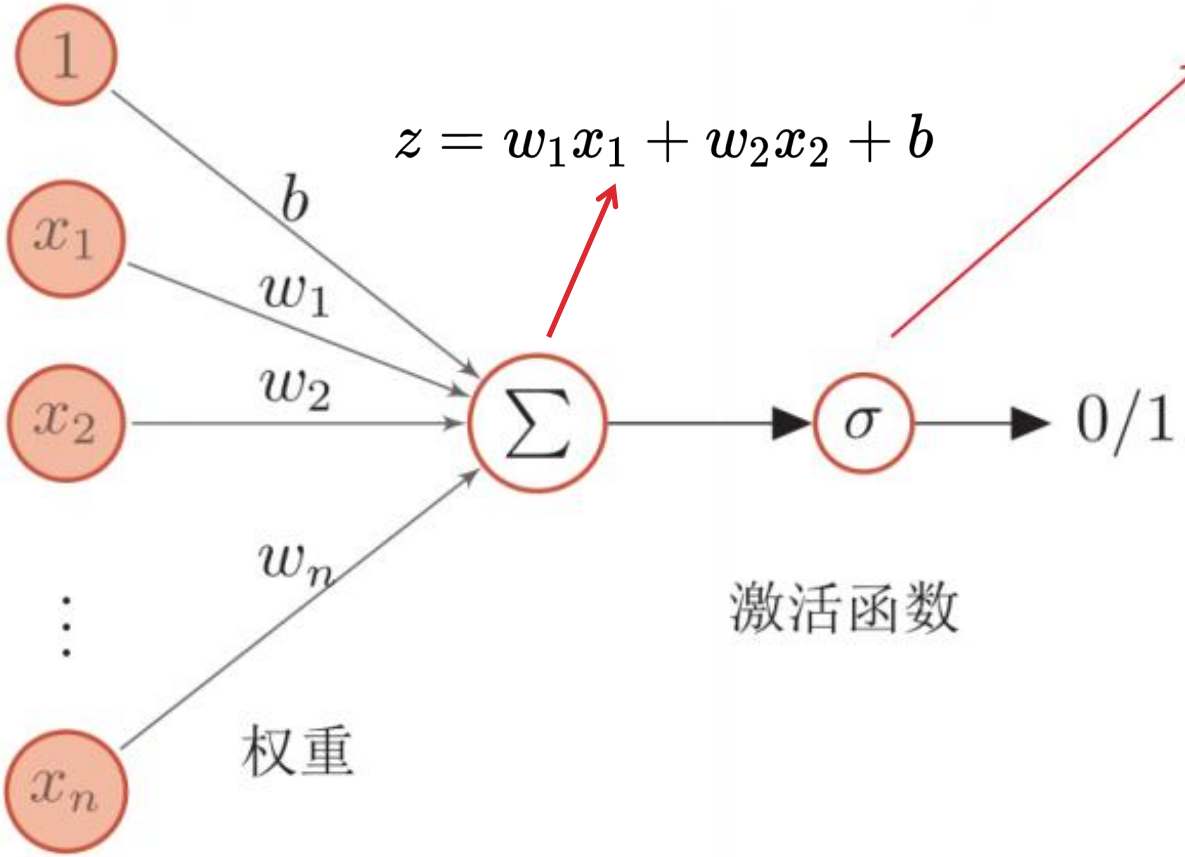
机器学习实验课3

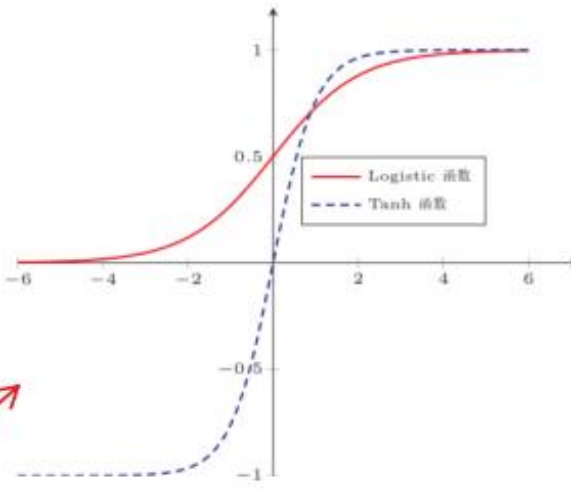
神经网络与卷积神经网络

ML02

致谢：梁威鹏

神经网络中的神经元


1
x_1
x_2
x_n
b
w_1
w_2
w_n
$\sum$
σ
0/1
$z = w_1x_1 + w_2x_2 + b$
激活函数
权重

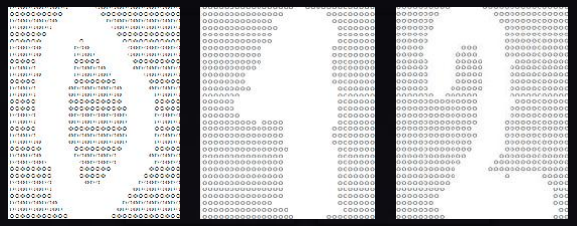

1
0.5
−0.5
−1
−6
−4
−2
2
4
6
Logistic 函数
Tanh 函数

神经网络学习——以手写数字为例

提问1：神经网络是如何识别出数字的?

回答：把一个识别任务转换为一个分类任务。

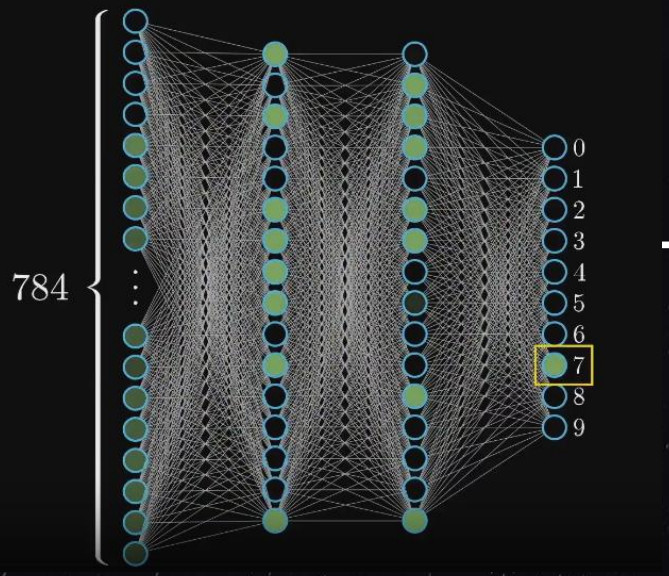
手写数字识别是一个多分类问题，共有10个分类，每个手写数字图像的 类别标签是0~9中的其中一个数。例如下面这三张图片的标签分别是0,1 2。





神经网络的输入:

28x28的图像



神经网络模型

[
0.1,
0.1,
0.05,
0.02,
0.05,
0.08,
0.05,
0.5,
0.03,
0.07,
]

one-hot vectors

神经网络输出

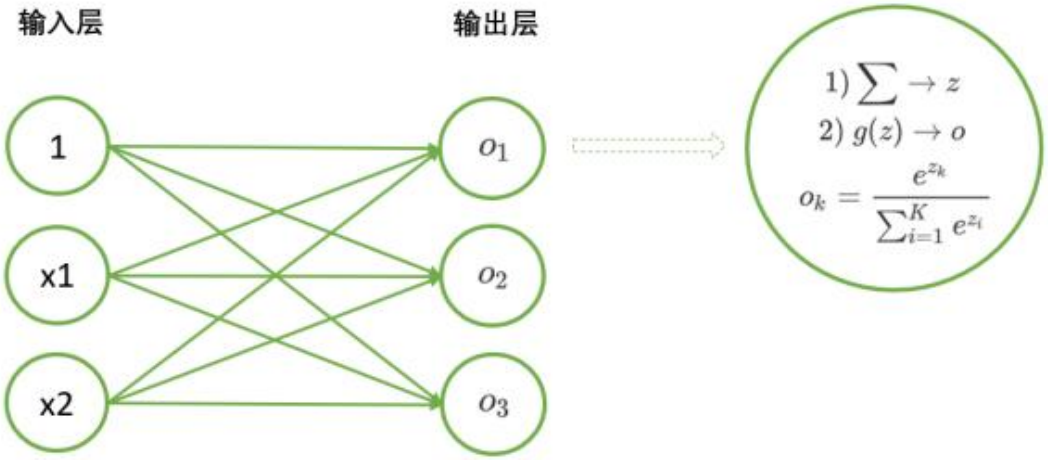
7

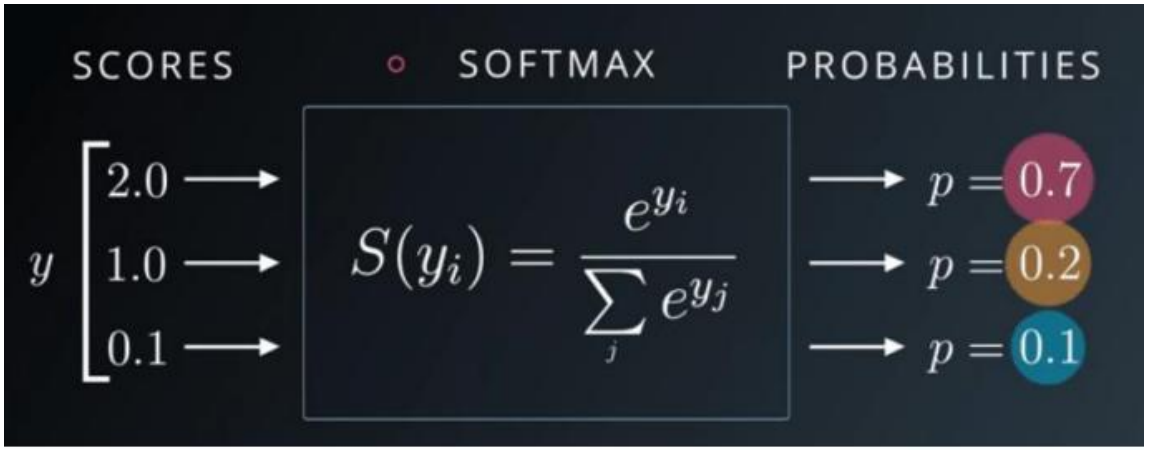
[
0,
0,
0,
0,
0,
0,
0,
1,
0,
0,
]

label

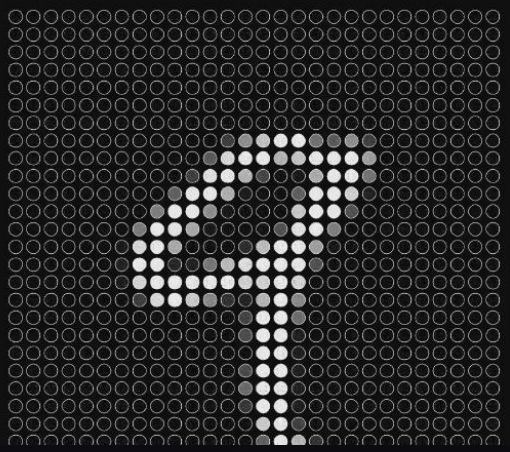
Softmax函数

Softmax函数是深度学习基础中的基础，它是神经网络进行多分类时，默认放在**输出层**中处理数据的函数。假设现在神经网络是用于三分类数据，且三个分类分别是苹果，柠檬和百香果，序号则分别是分类1、分类2和分类3。则使用softmax函数的神经网络的模型会如下所示：





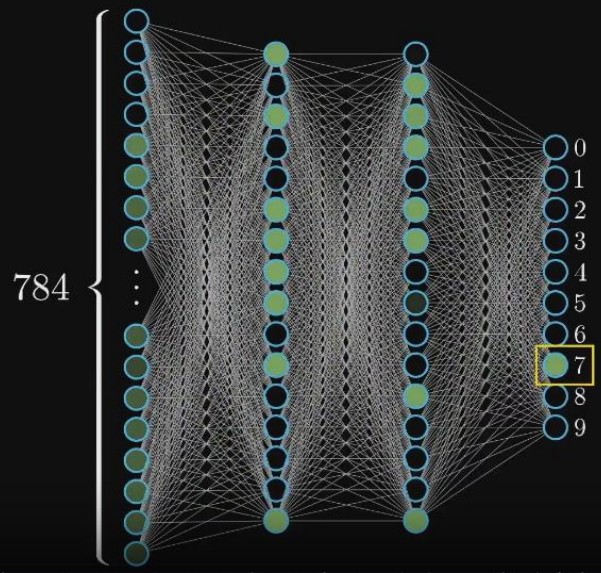
神经网络学习——以手写数字为例





神经网络的输入：

28x28的图像



神经网络模型

```
[
0.1,
0.1,
0.05,
0.02,
0.05,
0.08,
0.05,
0.5,
0.03,
0.07,
]
```

one-hot vectors

神经网络输出

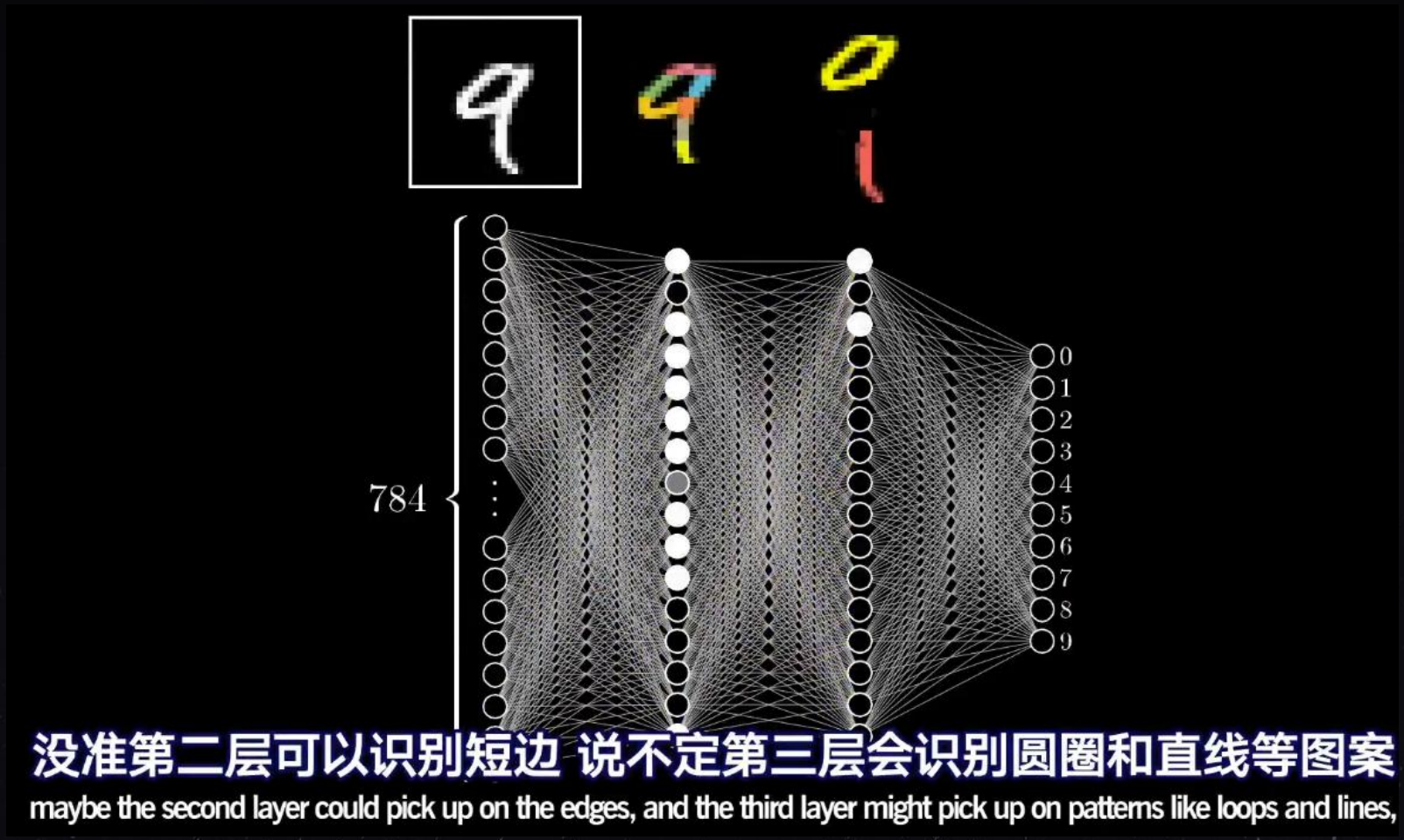
7

```
[
0,
0,
0,
0,
0,
0,
0,
1,
0,
0,
]
```

label

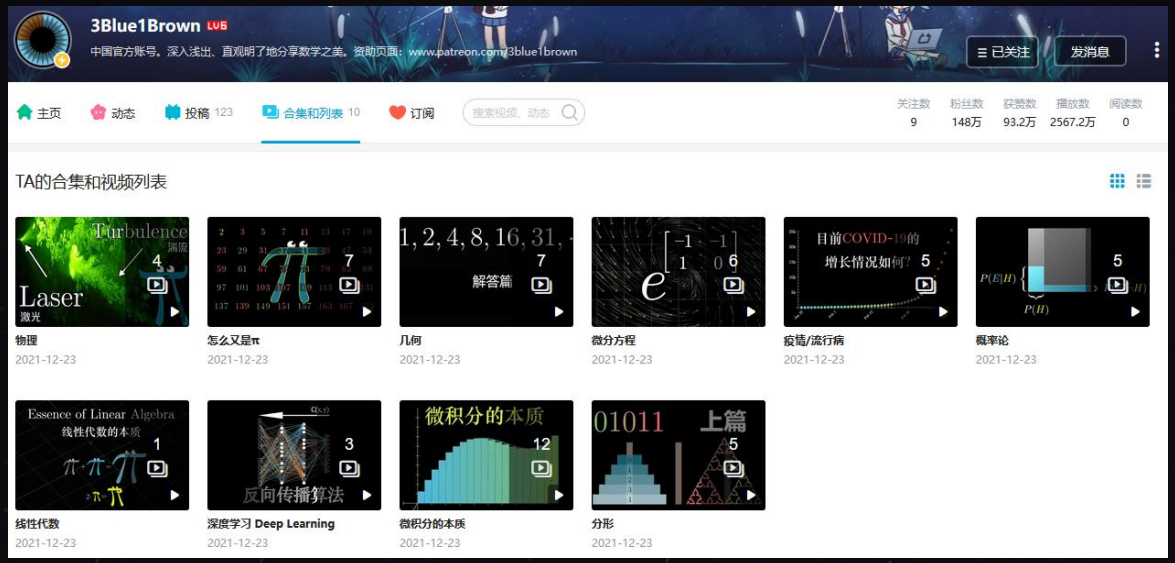
神经网络学习——以手写数字为例

提问2：为什么需要隐藏层?



神经网络基础巩固

- 【官方双语】深度学习之神经网络的结构 Part 1 传送门
- 【官方双语】深度学习之梯度下降法 Part 2 传送门
- 【官方双语】深度学习之反向传播算法 上/下 Part 3 传送门



传送门

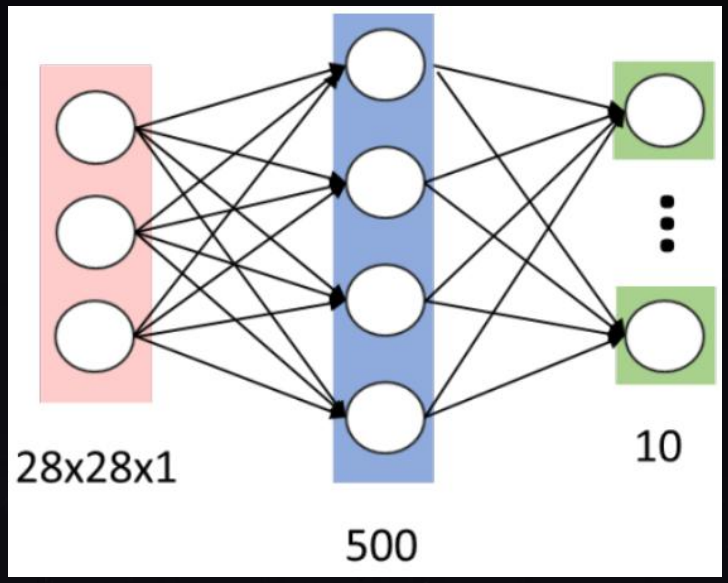
全连接网络的局限性

对于MNIST 手写数字识别，假如第一个隐层的节点数为500，那么一个全连接层的参数个数为：

- 28×28×1×500+500 ≈ 40万；

当图片更大时，比如CIFAR数据集中，图片大小为 32×32×3，此时全连接层的参数有：

- 32×32×3×500+500 ≈ 150万



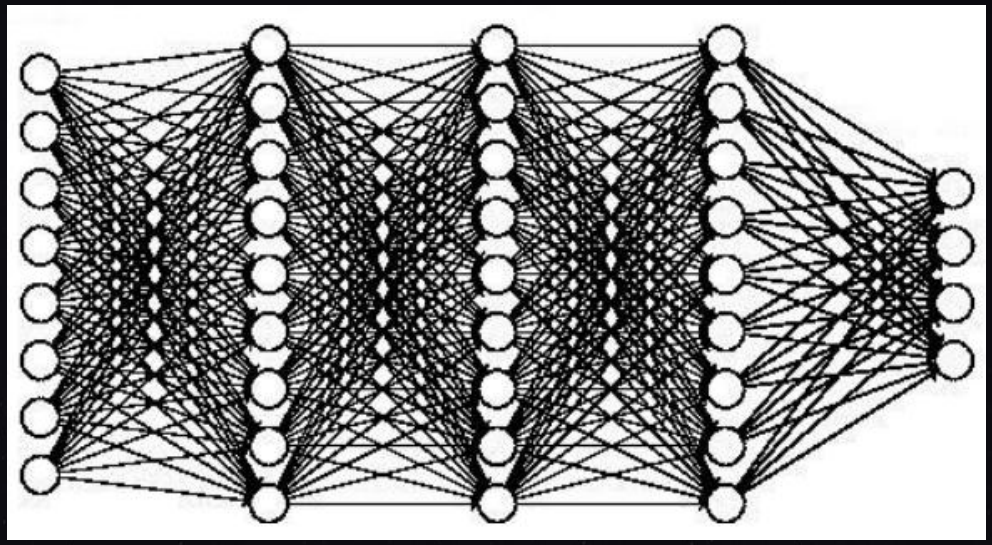
全连接网络的局限性

当图片分辨率进一步提高时，当隐层数量增加时，例如：600 x 600 图像，各隐层节点数分别为300,200和100，则参数个数为：

- 600 x 600 x 300 + 300 x 200 + 200 x 100 ≈ 1.08亿

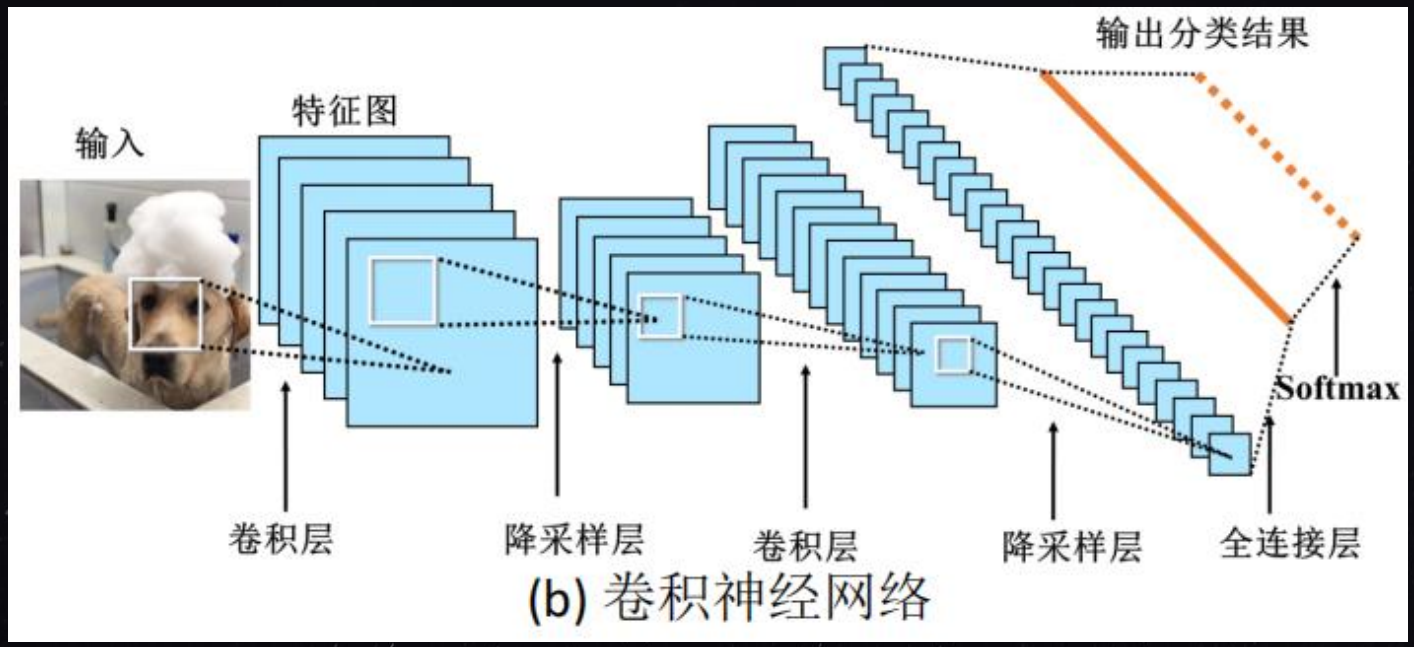
当参数增多会导致：

- 计算速度减慢
- 过拟合



卷积神经网络结构

- 输入层：将每个像素代表一个特征节点输入到网络中。
- 卷积层：卷积运算的主要目的是使原信号特征增强，并降低噪音。
- 降采样层：降低网络训练参数及模型的过拟合程度。
- 全连接层：对生成的特征进行加权。



LeCun et al., 1998

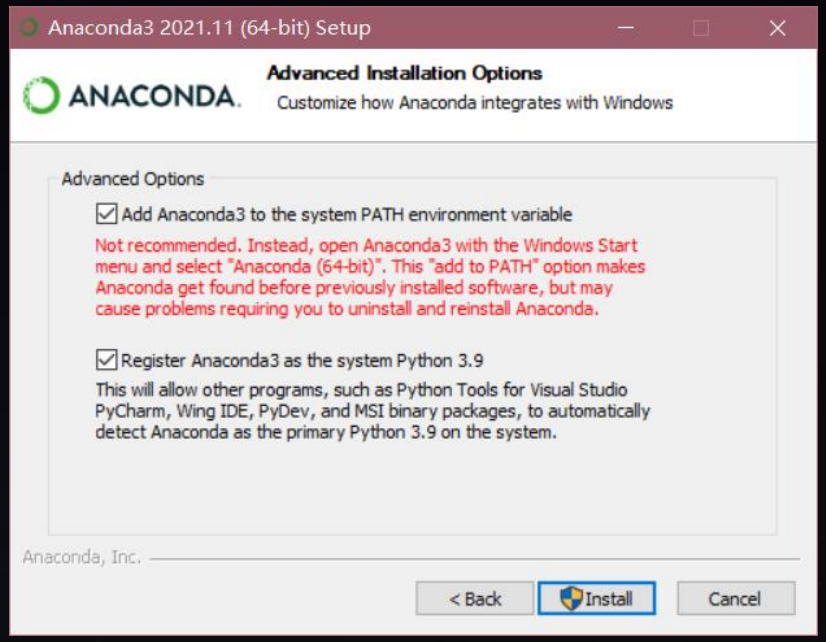
MindSpore介绍

- MindSpore官网课程 链接
- MindSpore官网初级课程 链接
- *MindSpore教程文档 链接
- MindSpore Gitee码云仓库 链接
- MindSpore代码快速入门：手写数字识别例子 链接
- 华为认证MindSpore高级工程师在线课程 链接

Anaconda安装

参考配置链接





conda添加源

PYTHON | 复制代码

```python
#查看当前conda配置
conda config --show channels

#设置通道
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/pytorch/

#设置搜索是显示通道地址
conda config --set show_channel_urls yes
```

创建Python环境参考

1. conda添加源（上一页ppt）

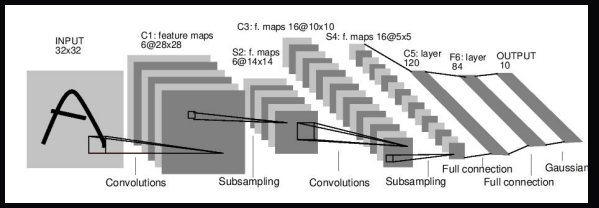
```
F:\MindSporeProject
(mindspore_cpu) λ pip install tqdm
```

2. > conda create --name mindspore python=3.7.5
3. > conda activate mindspore
4. > pip install tqdm
5. > pip install opencv-python
6. > pip install pyyaml
7. > pip install ml_collections

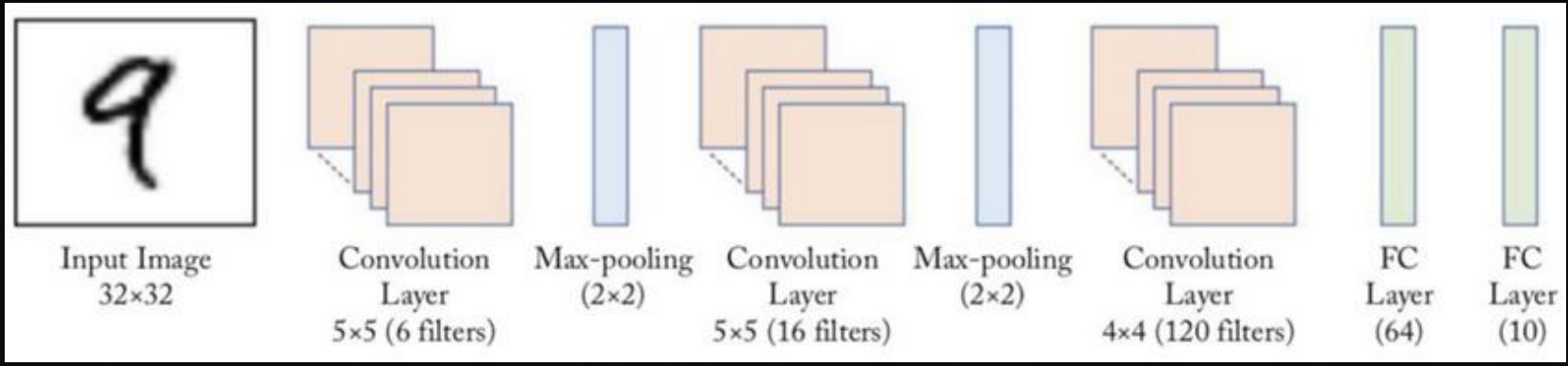


8. > pip install https://ms-release.obs.cn-north-4.myhuaweicloud.com/1.6.1/MindSpore/cpu/x86_64/mindspore-1.6.1-cp37-cp37m-win_amd64.whl --trusted-host ms-release.obs.cn-north-4.myhuaweicloud.com -i https://pypi.tuna.tsinghua.edu.cn/simple

LeNet介绍



LeNet是最早的**卷积神经网络**之一。1998年，Yan LeCun第一次将LeNet卷积神经网络应用到图像分类上，在手写数字识别任务中取得了巨大成功。LeNet通过**连续使用卷积和池化层的组合**提取图像特征。



- **第一轮卷积和池化**：卷积提取图像中包含的特征模式（激活函数使用sigmoid），图像尺寸从32减小到28。经过池化层可以降低输出特征图对空间位置的敏感性，图像尺寸减到14。

- **第二轮卷积和池化**：卷积操作使图像尺寸减小到10，经过池化后变成5。

- **第三轮卷积**：将经过第3次卷积提取到的特征图输入到全连接层。第一个全连接层的输出神经元的个数是64，第二个全连接层的输出神经元个数是分类标签的类别数，对于手写数字识别其大小是10。然后使用Softmax激活函数即可计算出每个类别的预测概率。

卷积和池化

卷积：padding = 0

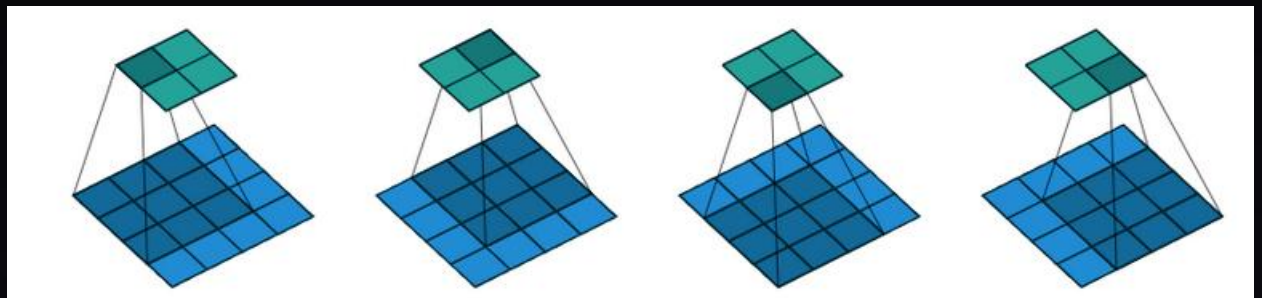
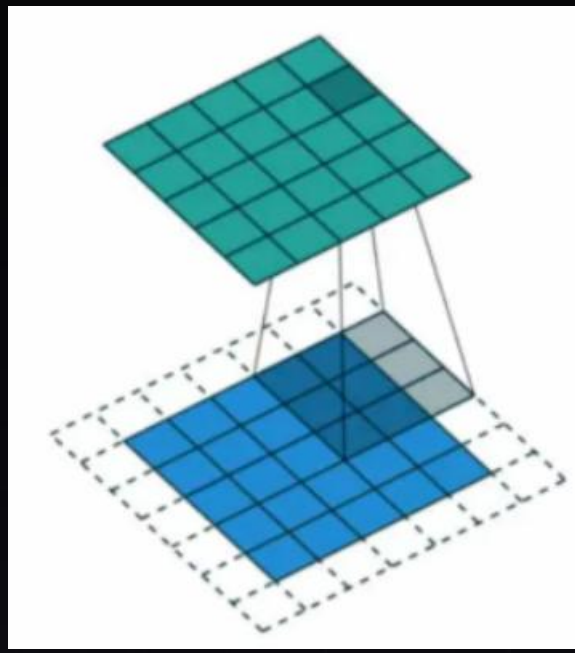


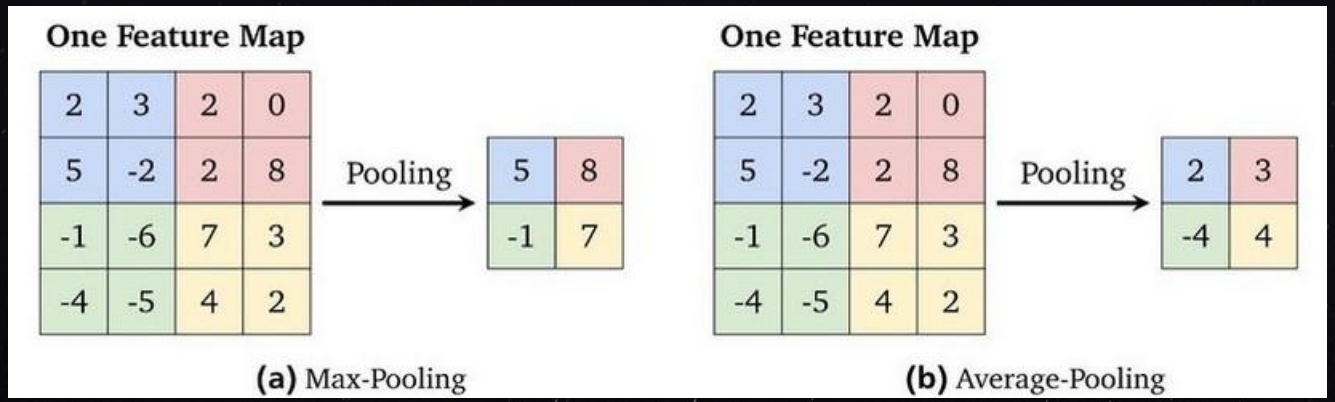
Figure 2.1: (No padding, unit strides) Convolving a 3×3 kernel over a 4×4 input using unit strides (i.e., $i = 4$, $k = 3$, $s = 1$ and $p = 0$).

GiantPandaCV

卷积：padding = 1



池化是一种down-sampling技术，本质是基于滑动窗口的思想，可以去除特征图中的**冗余信息**，降低特征图的维度。

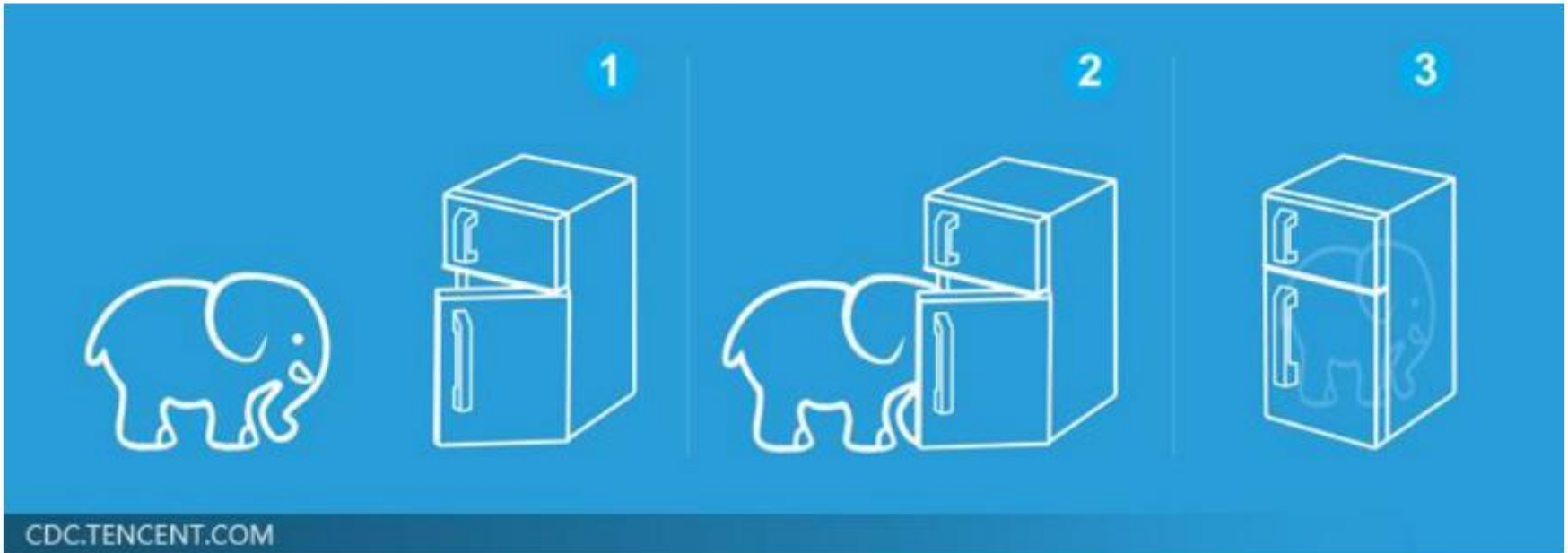


深度学习的三个步骤

来源：李宏毅《1天搞懂深度学



eep Learning is so simple ……



定义网络

使用MindSpore定义神经网络需要继承 **mindspore.nn.Cell**。

Cell是所有神经网络（如Conv2d-relu-softmax等）的**基类**。

神经网络的各层需要预先在__init__方法中定义，然后通过定义**construct**方法来完成神经网络的前向构造。

按照LeNet的网络结构，定义网络各层如下：

```
import mindspore.nn as nn
from mindspore.common.initializer import Normal

class LeNet5(nn.Cell):
    """
    Lenet网络结构
    """
    def __init__(self, num_class=10, num_channel=1):
        super(LeNet5, self).__init__()
        # 定义所需要的运算
        self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
        self.fc1 = nn.Dense(16 * 5 * 5, 120, weight_init=Normal(0.02))
        self.fc2 = nn.Dense(120, 84, weight_init=Normal(0.02))
        self.fc3 = nn.Dense(84, num_class, weight_init=Normal(0.02))
        self.relu = nn.ReLU()
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
```

```
    def construct(self, x):
        # 使用定义好的运算构建前向网络
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x
```

定义损失函数

损失函数用来评价模型的预测值和真实值不一样的程度。

mindspore.nn.loss也提供了许多其他常用的损失函数：

如绝对误差损失函数**SoftmaxCrossEntropyWithLogits**、**MSELoss**、**SmoothL1Loss**等。

我们给定输出值和目标值，计算损失值，使用方法如下所示：

```
import numpy as np

import mindspore.nn as nn
from mindspore import Tensor

loss = nn.L1Loss()
output_data = Tensor(np.array([[1, 2, 3], [2, 3, 4]]).astype(np.float32))
target_data = Tensor(np.array([[0, 2, 5], [3, 1, 1]]).astype(np.float32))
print(loss(output_data, target_data))
```

定义优化器

优化器用于计算和更新梯度，模型优化算法的选择直接关系到最终模型的性能，如果有时候效果不好，未必是特征或者模型设计的问题，很有可能是优化算法的问题。

MindSpore所有优化逻辑都封装在**Optimizer**对象中。API文档

需要构建一个Optimizer对象，这个对象能够保持当前参数状态并基于计算得到的梯度进行参数更新。

为了构建一个Optimizer，我们需要给它一个包含可优化的参数（必须是Variable对象）的迭代器，如网络中所有可以训练的parameter，将params设置为net.trainable_params()即可。然后，你可以设置Optimizer的参数选项，比如学习率、权重衰减等等。

```
# 定义优化器函数
from mindspore.nn as nn
net_opt = nn.SGD(network.trainable_params(), learning_rate=0.1, weight_decay=0.0)
```

准备数据集

在模型训练过程中，一般分为四个步骤。

1. 定义神经网络

2. 定义超参、损失函数及优化器

3. 读入数据集

```
# 下载并处理MNIST数据集
download_train = Mnist(path="./mnist", split="train", batch_size=32,
                       repeat_num=1, shuffle=True, resize=32, download=False)

download_eval = Mnist(path="./mnist", split="test", batch_size=32,
                      resize=32, download=False)

dataset_train = download_train.run()
dataset_eval = download_eval.run()
```

训练

在模型训练过程中，一般分为四个步骤。

1. 定义神经网络

2. 定义超参、损失函数及优化器

3. 读入数据集

4. 输入训练轮次和数据集进行训练。API文档

训练过程中会打印loss值，loss值会**波动**，但总体来说loss值会逐步**减小**，精度逐步**提高**。每个人运行的loss值有一定**随机**性，不一定完全相同。

```
# 训练及保存模型
# 在开始训练之前，MindSpore需要提前声明网络模型在训练过程中是否需要保存中间过程和结果，
# 因此使用`ModelCheckpoint`接口用于保存网络模型和参数，以便进行后续的Fine-tuning（微调）操作。
# 1. 设置模型保存参数
config_ck = CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)

# 2. 应用模型保存参数
ckpoint = ModelCheckpoint(prefix="lenet", directory="./lenet", config=config_ck)

# 3.通过MindSpore提供的`model.train`接口可以方便地进行网络的训练，`LossMonitor`可以监控训练过程中`loss`值的变化
# 初始化模型参数
model = Model(network, loss_fn=net_loss, optimizer=net_opt, metrics={'accuracy'})

# 4.训练网络模型
model.train(10, dataset_train, callbacks=[ckpoint, LossMonitor(0.01, 1875)])
```

验证集验证

在模型训练过程中，一般分为四个步骤。

1. 定义神经网络

2. 定义超参、损失函数及优化器

3. 读入数据集

4. 输入训练轮次和数据集进行训练。API文档

5. 验证集验证

可以在打印信息中看出模型精度数据，示例中精度数据达到**95%**以上，模型质量良好。随着网络迭代次数增加，模型精度会进一步提高。

```
# 5.通过模型运行测试数据集得到的结果，验证模型的泛化能力：
# 5.1 使用`model.eval`接口读入测试数据集。
acc = model.eval(dataset_eval)
print("{}".format(acc))
```

测试例子可视化

我们使用生成的模型进行一个batch图片数据的分类预测，具体步骤如下：

1. 加载保存的模型权重

2. 把模型权重赋予到网络的每个权重中

3. 加载测试集

4. 可视化该测试集

5. 使用model.predict()对该测试集进行预测

6. 结果对比

被预测的图片会随机生成，每次运行结果可能会不一样。

```
# 5.2 使用保存后的模型参数进行推理。
# ## 加载模型
# 加载已经保存的用于测试的模型
param_dict = load_checkpoint("./lenet/lenet-1_1875.ckpt")
# 加载参数到网络中
load_param_into_net(network, param_dict)

#加载测试集
mnist = Mnist("./mnist", split="test", batch_size=6, resize=32)
dataset_infer = mnist.run()
ds_test = dataset_infer.create_dict_iterator()
data = next(ds_test)
images = data["image"].asnumpy()
labels = data["label"].asnumpy()
```

测试例子可视化

我们使用生成的模型进行一个batch图片数据的分类预测，具体步骤如下：

1. 加载保存的模型权重

2. 把模型权重赋予到网络的每个权重中

3. 加载测试集

4. 可视化该测试集

5. 使用model.predict()对该测试集进行预测

6. 结果对比

```
# 可视化该测试集
plt.figure()
for i in range(1, 7):
    plt.subplot(2, 3, i)
    plt.imshow(images[i-1][0], interpolation="None", cmap="gray")
plt.show()
```

测试例子可视化

我们使用生成的模型进行一个batch图片数据的分类预测，具体步骤如下：

1. 加载保存的模型权重
2. 把模型权重赋予到网络的每个权重中
3. 加载测试集
4. 可视化该测试集
5. 使用model.predict()对该测试集进行预测
6. 结果对比

从打印结果可以看出，预测值与目标值大部分一致。

```
# 使用函数model.predict预测image对应分类
output = model.predict(Tensor(data['image']))
predicted = np.argmax(output.asnumpy(), axis=1)

# 输出预测分类与实际分类
print(f'Predicted: "{predicted}", Actual: "{labels}"')
```

实验任务

1. 使用MindSpore框架以及所给文档里的代码完成Mnist手写数据集的识别任务：

   1.1 训练模型；

   1.2 单个batch的测试集可视化，(batchsize = 8)；

   1.3 利用训练好的模型对单个batchsize进行预测，并给出它们的预测结果

2. 尝试通过调整模型结构 or 参数 or 超参数 or 权重初始化方式使得测试集准确率达到Baseline，即**准确率需要≥97%。** (还可观察其收敛时间长短)

具体请参考实验报告

